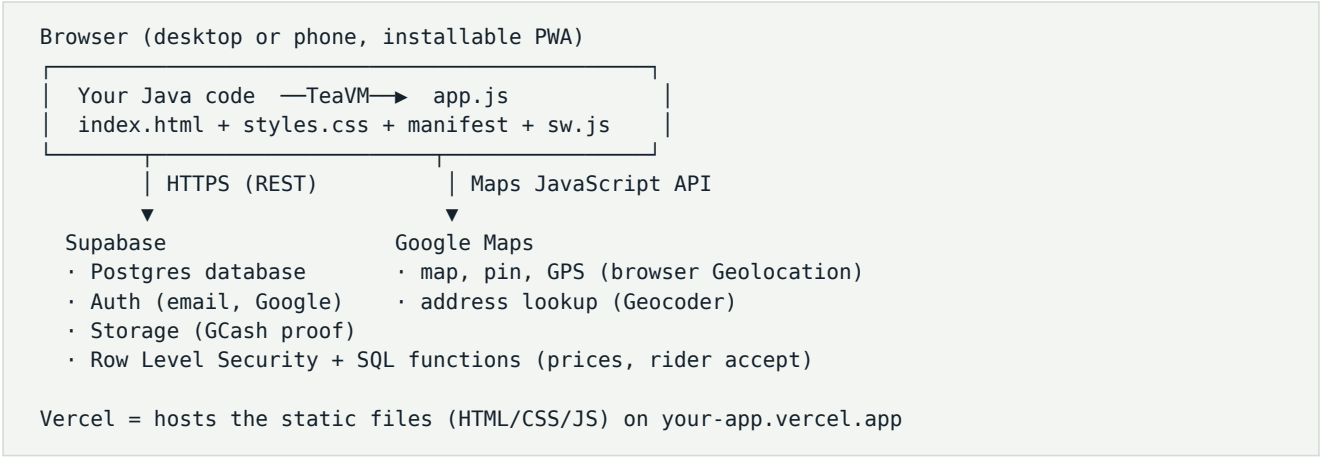
# Laundry HUB — Website

Desktop & mobile website, installable as an app · Java OOP (TeaVM) · Supabase · Vercel
Claude Code prompts, phase by phase · Service & API setup · VS Code only

## How it fits together

Vercel does not run Java on its servers. So the whole app is written in **Java (OOP)** and compiled by **TeaVM** into JavaScript that runs in the browser. Vercel only hosts the finished files. The browser talks straight to **Supabase** (database, login, file storage). The site is a **PWA**, so users can install it on a computer or phone from the browser — that is the “downloadable” part.




### What “Java OOP only” means here

All application logic — models, prices, validation, pages, routing, API calls — is Java. A few files cannot be Java because browsers and databases require their own formats: **index.html** and **styles.css** (page shell and design), **manifest.webmanifest** and a ~20-line **sw.js** (needed to make the site installable), and **SQL** for the Supabase tables, security rules and two database functions. Tell your teacher this up front.

### Payments: GCash without a server

Real-time GCash gateways (PayMongo) need a secret key on a server, which a Java-only static site cannot keep safe. So the design uses the method many small shops use: the customer sends the amount to the shop’s GCash number and enters the **13-digit reference number** (optionally uploading a screenshot). The admin verifies it in the Orders page. Cash is paid on delivery or at the counter. (An optional PayMongo upgrade is at the end of this guide.)

## Tech stack

| Part | Technology |
|---|---|
| Language | Java 21 (OOP), built with Maven in VS Code |
| Java → browser | TeaVM (Maven plugin) + JSO for calling browser APIs (DOM, fetch, Geolocation, Google Maps) |

| Part | Technology |
|---|---|
| UI | Your own Java component classes that create DOM elements; one styles.css with the design tokens; responsive (phone, tablet, desktop) |
| Database | Supabase Postgres, called through its REST API (PostgREST) from a Java SupabaseClient class |
| Login | Supabase Auth: email + password, and Google sign-in |
| Security | Row Level Security policies per role; SQL functions for price calculation and rider job accept |
| Files | Supabase Storage bucket for GCash payment screenshots |
| Maps | Google Maps JavaScript API (map, marker, Geocoder) + browser Geolocation (GPS) |
| Installable | PWA: web app manifest + service worker + app icons |
| Hosting | Vercel (static files); deploy from VS Code with the Vercel CLI or GitHub |
| Tests | JUnit 5 on the normal JVM for models and price logic |

# Design package

Unzip **Laundry_HUB_UI_Design.zip** into **docs/design/** in your project before Phase 0. Every prompt tells Claude Code to match it.

| Path | Contents |
|---|---|
| docs/design/DESIGN_SPEC.md | Tokens, navigation, every screen, desktop/tablet/phone rules (section 4), build rules |
| docs/design/screens/*.png | Screenshot of all 21 screens |
| docs/design/prototype/*.html | Clickable version of each screen; inline styles give exact values |
| docs/design/screens.json | List of screens (used for the design review) |

# Part A — Set up your tools and services

Do Part A once, before running the prompts. Everything is done in a browser or in the VS Code terminal.

## A1. Install on your computer

1. **JDK 21** (Eclipse Temurin, adoptium.net). Tick “Set JAVA_HOME”.
2. **Maven** (maven.apache.org) — or let Claude Code add the Maven wrapper (mvnw) in Phase 0.
3. **Node.js LTS** (nodejs.org) — needed for Claude Code, the Vercel CLI and the Supabase CLI. No Node code is written.
4. **Git**, and a free **GitHub** account.
5. VS Code extensions: **Extension Pack for Java**, **Live Server** (to preview the site), optionally **SQLTools** with the PostgreSQL driver.
6. Claude Code: `npm install -g @anthropic-ai/claude-code`, then run `claude` in the VS Code terminal.

## A2. Supabase project

1. Go to **supabase.com** → sign in with GitHub → **New project**. Name: laundry-hub. Choose a strong database password and save it. Region: **Southeast Asia (Singapore)** for speed in the Philippines.
2. When it finishes, open **Project Settings → API Keys**. Copy the **Project URL** (https://xxxx.supabase.co) and the **publishable key** (starts with `sb_publishable_`). Both are safe to use in the website. Supabase is replacing the old “anon” key with this publishable key.
3. Do **not** put the **secret key** (`sb_secret_`) anywhere in the website. Supabase rejects secret keys used from a browser, and it would bypass all security rules.
4. **Authentication → Sign In / Providers → Email**: keep it enabled. For school testing you may turn off “Confirm email”, then turn it back on before you present.
5. **Authentication → URL Configuration**: Site URL = http://127.0.0.1:5500 for now (Live Server). Add redirect URLs http://127.0.0.1:5500/** and later https://YOUR-APP.vercel.app/**.
6. **Storage → New bucket**: name `gcash-proofs`, Private. (Phase 1 adds the access rules.)
7. Tables and rules are created in Phase 1. Claude Code writes the SQL into supabase/migrations/. To apply it, open **SQL Editor → New query**, paste the file, click **Run**. (Or use the CLI: `npx supabase login`, `npx supabase link --project-ref YOUR_REF`, `npx supabase db push`.)
8. After running Phase 1, create your admin: sign up on the site with your email, then in **Table Editor → profiles** change your row’s role to `ADMIN`. Do the same with a rider account (role `RIDER`).

## A3. Google Cloud: Maps and Google sign-in

1. **console.cloud.google.com** → New Project “laundry-hub” → link a billing account. Billing is required even though the Maps JavaScript API includes free monthly usage per API (check the current pricing page). Set **Billing → Budgets & alerts** to a small amount with email alerts.
2. **APIs & Services → Library** → enable **Maps JavaScript API**, **Geocoding API** (used by the JS Geocoder) and optionally **Places API (New)** for address search.

3. **Credentials → Create credentials → API key** → name it laundryhub-web. Application restriction: **Websites** → add `http://127.0.0.1:5500/*`, `http://localhost:5500/*` and later `https://YOUR-APP.vercel.app/*`. API restriction: only the APIs you enabled. This key is visible in the website, so the website restriction is what protects it.

4. GPS needs no key: the browser's Geolocation API is used. It only works on **https** or on localhost/127.0.0.1.

5. Google sign-in: **APIs & Services → OAuth consent screen** → External → app name Laundry HUB, your email → add test users. Then **Credentials → Create credentials → OAuth client ID → Web application**. Authorized redirect URI: `https://YOUR-PROJECT.supabase.co/auth/v1/callback` (shown in Supabase under Providers → Google). Copy the Client ID and Client secret.

6. In Supabase: **Authentication → Sign In / Providers → Google** → enable, paste the Client ID and Client secret → Save. The Google secret lives only in Supabase, never in your code.

## A4. Vercel

1. Sign up at **vercel.com** with GitHub (Hobby plan is free).

2. In the VS Code terminal: `npm install -g vercel`, then `vercel login`.

3. Vercel's build machines do not have Java, so you build on your computer and upload the finished folder (Phase 9 sets this up): `mvnw clean package` → `cd web-dist` → `vercel --prod`. The first time, answer the questions (new project, name laundry-hub, no build command).

4. Optional: Phase 9 also creates a GitHub Actions workflow that installs Java, builds and deploys to Vercel on every push. It needs the GitHub secrets VERCEL_TOKEN, VERCEL_ORG_ID and VERCEL_PROJECT_ID (Vercel → Account Settings → Tokens; the IDs are in .vercel/project.json after the first deploy).

5. After the first deploy, copy your https://YOUR-APP.vercel.app address and add it to: the Google Maps key website list, Supabase Site URL and redirect URLs.

## A5. Where each value goes

| Value | Public? | Where |
|---|---|---|
| SUPABASE_URL | Yes | config.properties → copied into config.js at build time |
| SUPABASE_PUBLISHABLE_KEY | Yes (protected by RLS) | config.properties → config.js |
| GOOGLE_MAPS_KEY | Yes (website-restricted) | config.properties → config.js |
| Supabase secret key | NO | Never in the project. Only used by you in the dashboard if ever. |
| Google OAuth client secret | NO | Only inside Supabase → Providers → Google |
| Database password | NO | Your password manager; only for the CLI |

Add `config.properties` to .gitignore anyway, and commit a `config.example.properties` with empty values.

# Part B — Claude Code prompts

Run one phase at a time in Claude Code (VS Code terminal). Build and test each phase before the next. Phase 0 writes CLAUDE.md, which Claude Code reads automatically, so the rules apply to every phase.

## Phase 0 — Project setup, rules and design reference

```
Create a project for "Laundry HUB", a school OOP Java project: ONE responsive
website (desktop + mobile) that is installable as a PWA, written in Java and
compiled to JavaScript with TeaVM, using Supabase (REST API) as the database
and Vercel for static hosting. Development is VS Code only on Windows.

The UI design is already in docs/design (README.md, DESIGN_SPEC.md, screens/,
prototype/, screens.json, source/). Do NOT edit it. Read DESIGN_SPEC.md fully
(including section 4, desktop/tablet/phone rules) and look at every PNG first.

Set up:
- Maven project (add the Maven wrapper mvnw / mvnw.cmd), Java 21, TeaVM Maven
  plugin + teavm-classlib + teavm-jso + teavm-jso-apis (use the latest TeaVM
  release from Maven Central). Main class com.laundryhub.web.App. TeaVM output
  goes to web-dist/js/app.js. Static files live in src/main/webapp/ and are
  copied to web-dist/ during `mvnw package`.
- Packages: com.laundryhub.model, .service, .repository (Supabase access),
  .ui (components), .ui.pages, .util, .exception. JUnit 5 tests under
  src/test/java that run on the normal JVM (keep model/service code free of
  browser APIs so it is testable).
- config.properties (gitignored) with SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY,
  GOOGLE_MAPS_KEY; a build step writes them into web-dist/config.js
  (window.LH_CONFIG = {...}); commit config.example.properties.
- supabase/migrations/ folder for SQL. docs/ folder for documentation.

Write CLAUDE.md that every session must follow:
1. JAVA ONLY: all application logic (models, validation, prices, pages,
   routing, Supabase calls, maps) is Java compiled by TeaVM. The only non-Java
   files allowed: index.html (shell), styles.css, manifest.webmanifest, sw.js,
   icons, config.js (generated) and SQL migrations. No JS frameworks, no npm
   packages in the app.
2. DESIGN RULE: docs/design is the source of truth. Before building or
   changing a screen: open its PNG, read its HTML in docs/design/prototype for
   exact values, read its section in DESIGN_SPEC.md. Same text, order, colors,
   sizes and behavior. Follow DESIGN_SPEC.md section 4 for desktop/tablet/phone.
   After building, compare with the PNG at 390px and desktop width, list
   differences and fix them.
3. OOP: show Encapsulation, Inheritance, Polymorphism and Abstraction clearly,
   with "// OOP: <pillar>" comments. Private fields, validation in
   constructors, no magic numbers, Javadoc on public methods.
4. SECURITY: only the Supabase publishable key in the site; every table has
   Row Level Security; prices and rider job acceptance are decided in SQL
   functions, never trusted from the browser. Never commit config.properties.
5. Product rules: Walk-in or Pickup & Delivery; MINIMUM 8 KG; soaps Standard
   +₱0, Hypoallergenic +₱5, Premium Scented +₱8, Baby-Safe +₱6 per kg; price =
   kg × rate (default ₱30) + kg × soap add-on + delivery fee (₱60, delivery only);
   payment Cash or GCash (manual reference number, admin verifies). Roles:
   CUSTOMER, RIDER, ADMIN, STAFF. Order statuses: PENDING, WASHING, READY,
   OUT_FOR_DELIVERY, COMPLETED, CANCELLED. Each delivery order creates a PICKUP
   and a RETURN_DELIVERY rider job that on-duty riders accept or decline.
6. Copy the design tokens (DESIGN_SPEC.md section 1) and the screen list
   (screens.json) into CLAUDE.md.
7. End every phase with the exact commands to build and preview:
   `mvnw clean package`, then open web-dist with Live Server
   (http://127.0.0.1:5500), and `mvnw test`.

Also create: .gitignore, README.md, .vscode/settings.json and tasks.json
(tasks: Build, Test), src/main/webapp/index.html (loads styles.css, config.js,
the Google Maps script with the key from LH_CONFIG, then js/app.js), and copy
the fonts from docs/design/prototype/assets/fonts into src/main/webapp/fonts.
Build once and show me a "Hello Laundry HUB" page to prove TeaVM works.
```

## Phase 1 — Supabase database, security rules and SQL functions

```
Read CLAUDE.md. Write SQL migrations in supabase/migrations/ (one file per
step, numbered) that I will paste into the Supabase SQL Editor. Check every
screen in docs/design/screens.json so all data shown exists.
- Tables: profiles (id = auth.users.id, full_name, mobile, role, created_at),
  shop_settings (single row: rate_per_kg, min_kg, delivery_fee,
  delivery_radius_km, shop_lat, shop_lng, shop_address, shop_hours,
  gcash_number, gcash_account_name, cash_enabled, gcash_enabled), soaps (name,
  description, add_on_per_kg, available), orders (order_no like LH-0001,
  customer_id, service_type, kg, soap_id, time_slot, lat, lng, address,
  address_details, payment_method, payment_status UNPAID/PENDING_VERIFICATION/
  PAID/REJECTED, gcash_reference, gcash_proof_path, laundry_cost, soap_cost,
  delivery_fee, total, status, rider_id, created_at), order_status_history,
  riders (profile_id, plate_no, on_duty, last_lat, last_lng, last_seen),
  rider_jobs (order_id, type PICKUP/RETURN_DELIVERY, status AVAILABLE/ACCEPTED/
  IN_PROGRESS/COMPLETED/DECLINED, rider_id, current_step, accepted_at,
  completed_at, version).
- Trigger: on new auth user, insert a profiles row with role CUSTOMER.
- Helper function current_role() reading profiles.role for auth.uid().
- Row Level Security on EVERY table: customers see/insert only their own
  orders; riders see AVAILABLE jobs (when on duty) and their own jobs and the
  orders linked to them; ADMIN/STAFF see and update everything; soaps and
  shop_settings readable by everyone, writable by ADMIN only.
- SQL functions (security definer, called via /rest/v1/rpc):
  quote_order(service_type, kg, soap_id) returns the breakdown;
  place_order(...) validates kg >= min_kg (or charges as min), delivery pin
  required and inside delivery_radius_km (Haversine), recalculates all prices
  from shop_settings/soaps (ignores prices from the browser), creates the
  order + status history + the two rider jobs for delivery;
  accept_job(job_id) uses UPDATE ... WHERE status = 'AVAILABLE' so only one
  rider can win (returns an error the app shows as "already taken");
  advance_job(job_id) moves to the next step and updates order status
  (PICKUP done -> WASHING; RETURN_DELIVERY started -> OUT_FOR_DELIVERY;
  delivered -> COMPLETED); admin_dashboard() returns the counts and totals
  for screen A1; admin_reports(from, to) for screen A5.
- Storage policies for bucket gcash-proofs: a customer can upload to
  their own folder (user id), ADMIN/STAFF can read all.
- Seed: shop_settings defaults, the 4 soaps, sample orders like screen A2.
Then write docs/DATABASE.md with an ERD (Mermaid) and the steps to run the
files in the SQL Editor, plus how to make a user ADMIN or RIDER.
```

## Phase 2 — Java core: OOP model, Supabase client, router and components

```
Read CLAUDE.md.
- Domain model (pure Java, unit tested): abstract class User -> Customer,
  Rider, Admin; abstract ServiceType with getServiceFee() -> WalkInService,
  PickupDeliveryService; interface Payable + abstract PaymentMethod ->
  CashPayment, GCashPayment (validates 13-digit reference); abstract RiderJob
  with getSteps()/getActionLabels() -> PickupJob, DeliveryJob (steps and
  labels exactly as DESIGN_SPEC.md; cash step only for cash orders);
  Location (lat, lng, address); LaundryLoad (MIN_KG = 8, throws
  InvalidWeightException); Order; PriceCalculator (used for the live estimate
  in the UI; the database recalculates the real price).
- JUnit tests: 10 kg Hypoallergenic delivery = ₱410.00, 12 kg Premium Scented
  delivery = ₱516.00, 8 kg Standard walk-in = ₱240.00, 7 kg -> charged as 8.
- Infrastructure (TeaVM JSO): HttpClient wrapper around XMLHttpRequest/fetch
  with JSON; SupabaseClient with auth (sign up, sign in with password, sign
  in with Google via /auth/v1/authorize redirect, refresh token, sign out,
  session kept in localStorage), rest (select/insert/update with filters),
  rpc(name, params) and storage upload. Repositories per table on top of it
  (OrderRepository, SoapRepository, RiderJobRepository, ...) behind
  interfaces (OOP: abstraction).
- UI framework: abstract class Component (render(), mount, unmount), abstract
  class Page extends Component (title, requiresRole), Router with hash routes
  (#/signin, #/home, #/order/service, ... #/rider, #/admin/orders) and role
  guards; reusable components matching the design: Button (primary, outline,
  dark), Card, SelectableCard, TextInput, Stepper, Chip, StatusBadge,
  SegmentedControl, BottomNav, TopNav (desktop), Sidebar (admin), Toast,
  Spinner, EmptyState. Responsive layout helper using a matchMedia listener for
  the breakpoints in DESIGN_SPEC.md section 4.
- styles.css: every design token as a CSS variable, the component classes,
  and the three breakpoints. Fonts from /fonts.
Build and show a component gallery page (#/dev/components) that I can compare
with the design.
```

## Phase 3 — Sign in, sign up and role routing

```
Read CLAUDE.md. Design: docs/design/screens/00a-sign-in.png, 00b-sign-up.png,
00c-admin-sign-in.png and their prototype HTML.
- Sign in page with Customer / Rider / Admin segmented control (heading,
  email label and Google button visibility change like the prototype), email,
  password, remember me, forgot password (Supabase reset email), Sign in,
  "or", Continue with Google (customers), Sign up link.
- Sign up page with full name, mobile (09XXXXXXXXX), email, password, confirm,
  terms checkbox; saves full_name and mobile to profiles.
- After sign in, read profiles.role and route: CUSTOMER -> #/home, RIDER ->
  #/rider, ADMIN/STAFF -> #/admin. If the chosen tab does not match the real
  role, show a friendly message.
- Desktop: center the sign-in card (max 440px) on the light background with
  the blue header area; phone: exactly the mockup.
Compare with the PNGs (phone and desktop width) and list what you fixed.
```

## Phase 4 — Customer pages and order wizard

```
Read CLAUDE.md. Design: 01-home.png, 02-choose-service.png,
03-weight-and-soap.png, 04b-drop-off-walk-in.png, 05-review-and-pay.png,
06-order-placed.png and the prototype HTML (copy the behavior of
docs/design/prototype/01-home.html).
- Home, then the 4-step wizard with back button, "Step N of 4", progress bar,
  live estimate (PriceCalculator, confirmed with rpc quote_order), and a
  Continue / Place order button disabled until the step is valid.
- Service cards, weight stepper (minus disabled at 8) with warning note, soap
  cards loaded from the soaps table (available only), time-slot chips,
  walk-in shop card (from shop_settings), Review & pay summary, Cash / GCash.
- GCash: show the shop GCash number, account name and amount from
  shop_settings, a 13-digit reference input and optional screenshot upload
  (Supabase Storage, gcash-proofs/<user id>/<order no>.jpg). Payment status
  becomes PENDING_VERIFICATION.
- Place order calls rpc place_order and shows Order placed.
- Phone: bottom bar + bottom nav as in the mockups. Desktop (>= 1200px): top
  nav, two columns with a sticky Order summary card on the right (section 4).
Compare with the PNGs at 390px and 1280px and list what you fixed.
```

## Phase 5 — Google Maps pin and GPS

```
Read CLAUDE.md. Design: 04a-pin-location-delivery.png and its HTML.
- Write JSO bindings in Java for the parts of the Google Maps JavaScript API
  you need (Map, Marker or AdvancedMarkerElement, LatLng, event listeners,
  Geocoder) and for navigator.geolocation. No JavaScript files.
- Delivery location step: same layout as the mockup (hint, rounded map,
  status chip "Tap anywhere to pin" -> "Location pinned", Address details,
  Pickup time chips). Tapping the map moves the orange pin; the pin can be
  dragged; "Use my current location" button uses Geolocation (handle
  permission denied and http errors with friendly messages).
- After each move, reverse-geocode with the Geocoder to fill the address.
- Warn early if the pin is outside delivery_radius_km (the database checks
  again in place_order).
- Continue requires a pin and a time slot.
```

## Phase 6 — Order tracking and order history

```
Read CLAUDE.md. Design: 07-track-order.png and its HTML.
- My orders page: list of the customer's orders with StatusBadge colors,
  total and payment status (GCash: "Waiting for verification" / "Paid" /
  "Rejected — please resend").
- Track order page like the mockup: Google Map with shop, rider (from
  riders.last_lat/last_lng) and customer markers, rider card with call link
  (tel:), and the 5-step status timeline from order_status_history.
  Refresh every 10 seconds while the page is open (stop when leaving).
Compare with the PNG at phone and desktop width.
```

## Phase 6B — Rider pages: accept bookings

```
Read CLAUDE.md. Design: R1-rider-bookings.png, R2-rider-pickup-job.png,
R3-rider-delivery-cash.png, R4-rider-job-done.png, R5-rider-off-duty.png and
docs/design/prototype/R1-rider-bookings.html (copy its exact behavior).
- Rider home: dark header, greeting, On duty / Off duty switch (updates
  riders.on_duty), counters, tabs Available / My jobs, booking cards with
  type badge, order no. + distance (Haversine from the rider's GPS),
  customer, address, slot + kg, payment strip, Decline / Accept booking.
  Refresh every 15 s; off-duty and empty states.
- Accept calls rpc accept_job; if another rider won, show "This booking was
  already taken." Then open the Active job page.
- Active job: map with markers and a "Navigate in Google Maps" link
  (https://www.google.com/maps/dir/?api=1&destination=LAT,LNG), customer card
  with tel: link, step checklist and one big button (labels from
  PickupJob/DeliveryJob), rpc advance_job on each tap; cash orders get the
  "Cash received" step which sets payment_status PAID.
- Job completed page, then back to bookings.
- While on duty and the page is open, send the GPS position to riders
  (last_lat, last_lng, last_seen) every 10 seconds using watchPosition.
- Desktop: bookings list left, active job map right (section 4).
```

## Phase 7 — Admin pages

```
Read CLAUDE.md. Design: A1-admin-dashboard.png, A2-admin-orders.png,
A3-admin-pricing.png, A4-admin-users.png, A5-admin-reports.png and their
prototype HTML (read the inline styles for exact values).
- Admin layout: dark sidebar with logo, nav (active item style), profile and
  Log out; kicker + title, search, bell. Phone: sidebar becomes a top bar with
  a menu button (section 4).
- Dashboard (rpc admin_dashboard), Orders (filter chips with counts, list +
  detail panel with Google Map of the pin, status buttons that insert into
  order_status_history, assign rider dropdown, price breakdown; for GCash show
  reference no., screenshot preview from Storage and Verify payment / Reject
  buttons), Pricing & soaps (edit shop_settings incl. GCash number and account
  name; add/edit/toggle soaps), Customers / Riders / Staff (change roles,
  add rider plate no.), Sales reports (rpc admin_reports, bars by soap,
  Export CSV generated in Java and downloaded with a Blob link).
Compare every page with its PNG and list what you fixed.
```

## Phase 8 — Installable app (PWA), responsive check and design review

```
Read CLAUDE.md.
- PWA: manifest.webmanifest (name "Laundry HUB", short_name "LaundryHUB",
  theme_color #1E63A8, background_color #F3F5F2, display standalone, start_url
  "/"), icons 192/512 and maskable versions generated from the logo in the
  design, a small sw.js that caches the app shell (index.html, styles.css,
  fonts, js/app.js, config.js) and never caches Supabase or Google requests.
  Register the service worker from Java. Add an "Install app" button that
  appears when the browser fires beforeinstallprompt, plus iPhone
  instructions ("Share → Add to Home Screen").
- RESPONSIVE: test every page at 390px, 768px and 1280px; nothing scrolls
  sideways except tables in their own container.
- DESIGN REVIEW: for each entry in docs/design/screens.json compare the built
  page with its PNG; write docs/DESIGN_REVIEW.md (screen | matches? |
  differences | fixed?) and fix every difference you can.
- Tests: run all JUnit tests; add missing ones. Check loading, empty and
  error states. Write docs/OOP.md (each pillar -> classes and lines, UML class
  diagram in Mermaid).
```

### Phase 9 — Deploy to Vercel

```
Read CLAUDE.md. Prepare deployment of web-dist to Vercel as a static site:
- vercel.json in web-dist (or project root with outputDirectory web-dist):
  no build command, SPA fallback to index.html, correct content types for
  manifest.webmanifest and sw.js, and Cache-Control: no-cache for sw.js and
  index.html.
- A script build-and-deploy.cmd: mvnw clean package, then `vercel --prod`
  inside web-dist (Vercel's build machines have no Java, so we upload the
  already-built files).
- Optional .github/workflows/deploy.yml: checkout, setup-java 21, write
  config.properties from GitHub secrets, mvnw package, then
  `npx vercel deploy --prod --token $VERCEL_TOKEN` from web-dist.
- docs/DEPLOYMENT.md: first deploy steps, adding the Vercel URL to the Google
  Maps key restrictions and to Supabase Site URL / redirect URLs, and a final
  checklist (sign in, order with map pin, GCash verify, rider accept,
  install on phone).
```

# Phase checklist

| Phase | You should have | Check before moving on |
|---|---|---|
| 0 | TeaVM project, CLAUDE.md, fonts | “Hello Laundry HUB” page opens in Live Server |
| 1 | Tables, RLS, SQL functions | SQL runs without errors; you can make yourself ADMIN |
| 2 | OOP model, SupabaseClient, components | JUnit tests pass; component gallery matches the design |
| 3 | Sign in / sign up | Each role lands on the right page |
| 4 | Order wizard | Live total; minus stops at 8 kg; order appears in Supabase |
| 5 | Map pin + GPS | Pin fills the address; outside-radius warning works |
| 6 | Tracking + history | Status changes made by admin show up within 10 s |
| 6B | Rider pages | Two riders cannot accept the same booking |
| 7 | Admin pages | GCash verify, status update and pricing changes work |
| 8 | PWA + design review | “Install app” works on desktop Chrome and on a phone |
| 9 | Live site on Vercel | Everything works on https://YOUR-APP.vercel.app |

# Optional: real GCash checkout (PayMongo)

If your teacher allows one small non-Java piece, PayMongo can confirm GCash automatically. It needs a server to keep the PayMongo secret key, which Supabase provides as an **Edge Function** (written in TypeScript). The function creates a Payment Intent with GCash allowed, returns the checkout link, and a second function receives the PayMongo webhook and marks the order PAID. Keep the manual reference-number method as the default if the project must be 100% Java.

# Troubleshooting

| Problem | Usual cause |
| --- | --- |
| Blank page after build | TeaVM error: run mvnw package and read the first error; check the browser console (F12) |
| 401 / “Invalid API key” from Supabase | Wrong publishable key in config.properties, or the key was sent in the Authorization header instead of apikey |
| Empty lists although data exists | Row Level Security policy missing for that role |
| “This page can’t load Google Maps correctly” | Current address (127.0.0.1:5500 or the Vercel URL) is not in the key’s website list |
| “Use my location” does nothing | Page not on https or localhost, or permission denied in the browser |
| Google sign-in returns to the wrong page | Site URL / redirect URLs not set in Supabase URL Configuration |
| Old version keeps showing after deploy | Service worker cache: bump the cache name in sw.js, hard refresh (Ctrl+Shift+R) |
| No “Install app” button | Site must be on https with a valid manifest, icons and a registered service worker |